# Advance Applications in Data Science - Introduction to PageRank Algorithm

Week # 15

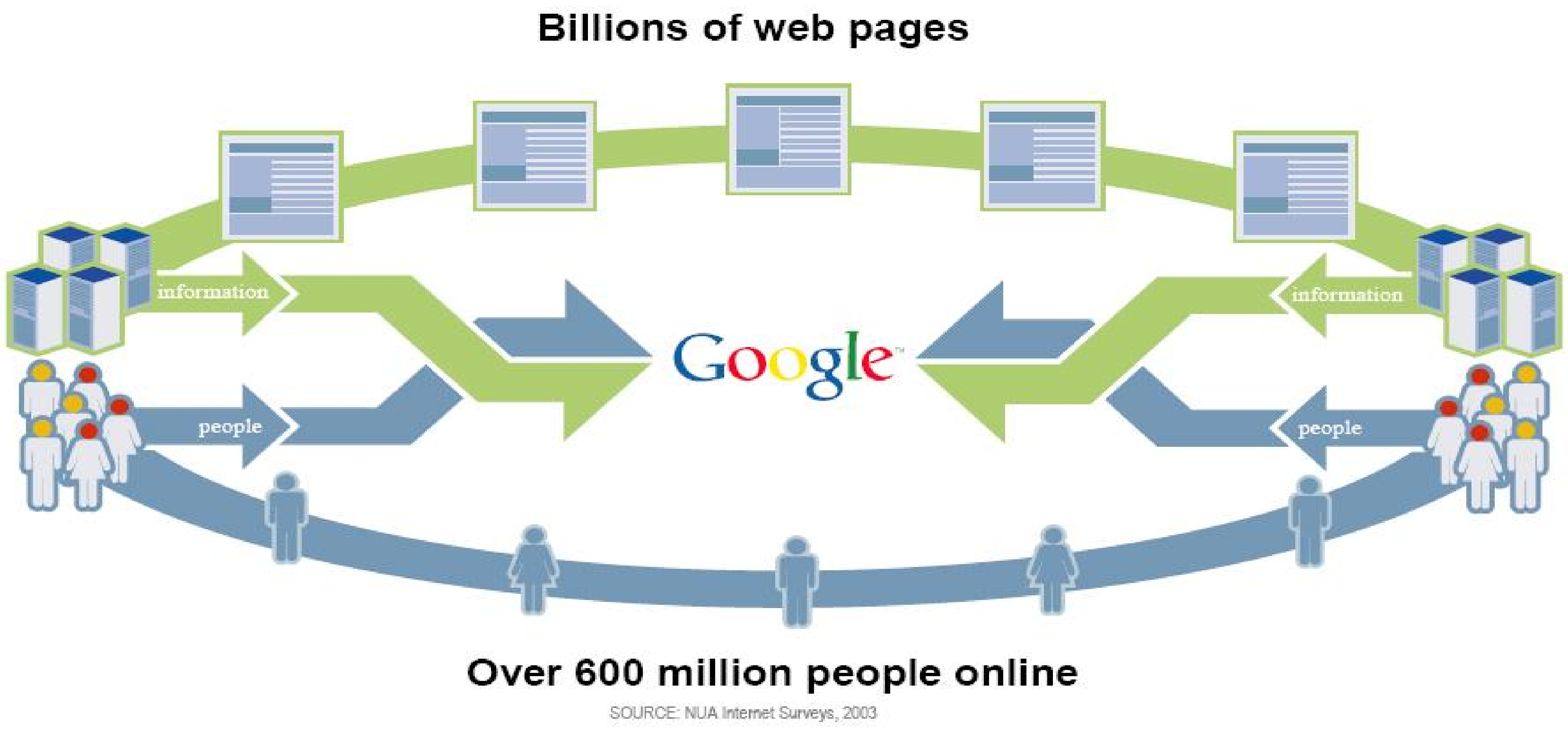
Course Instructor: Mahrukh Khan

- Many modern problems naturally form graphs:
  - Google Search
  - Social networks
  - Fraud detection
  - Recommendation systems
  - Transportation & logistics
  - Graphs capture relationships, connections, influence, not just values.

# What is Google?

- Google is a search engine owned by Google, Inc. whose mission statement is to "organize the world's information and make it universally accessible and useful". The largest search engine on the web, Google receives over 200 million queries each day through its various services.


# Introduction

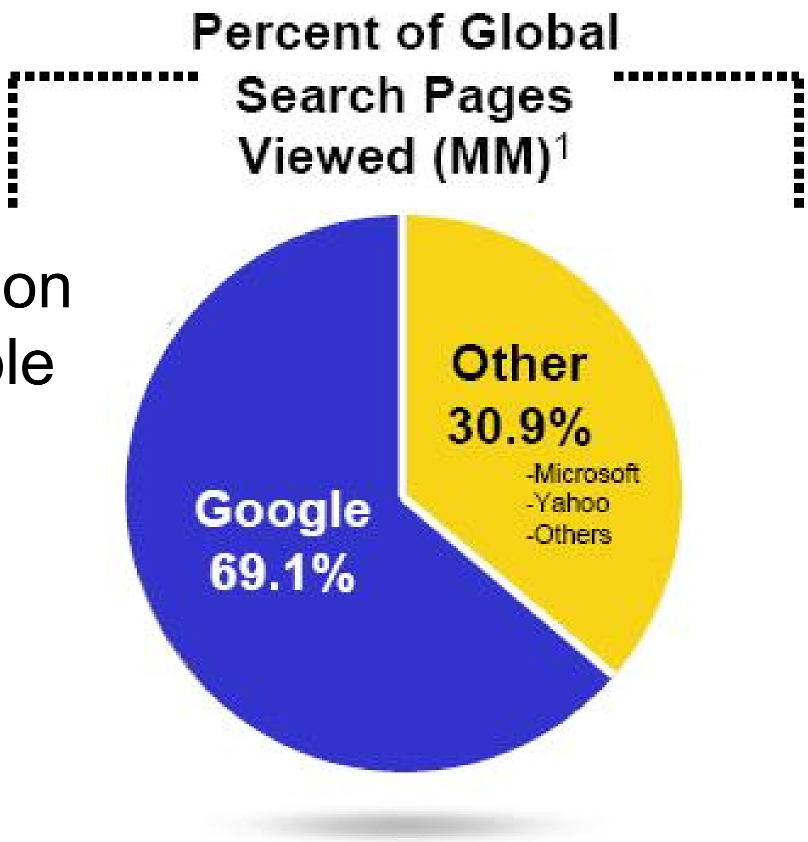


# Introduction

- Google's Mission

  To organize the world's information and make it universally accessible and useful

- Scaling with the web
  - Improved Search Quality
  - Academic Search Engine Research



# Motivation and Introduction

- Why is Page Importance Rating important?
  - New challenges for information retrieval on the World Wide Web.
  - Huge number of web pages: 150 million by1998
    1000 billion by 2008
  - Diversity of web pages: different topics, different quality, etc.

- What is PageRank?
  - A method for rating the importance of web pages objectively and mechanically using the link structure of the web.

# Background

- History:
  - Proposed by Sergey Brin and Lawrence Page (Google's Bosses) in 1998 at Stanford.
  - Algorithm of the first generation of Google Search Engine.
  - Shortly after, Page and Brin founded Google.
  - "The Anatomy of a Large-Scale Hypertextual Web Search Engine".
- Target:
  - Measure the importance of Web page based on the link structure alone.
  - Assign each node a numerical score between 0 and 1: PageRank.
  - Rank Web pages based on PageRank values.

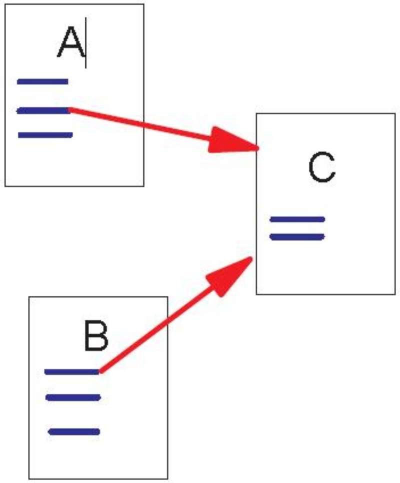
# PageRank

- PageRank is a link analysis algorithm which assigns a numerical weighting to each Web page, with the purpose of "measuring" relative importance.

  - Based on the hyperlinks map
  - An excellent way to prioritize the results of web keyword searches

# Link Structure of the Web

- 150 million web pages → 1.7 billion links




Backlinks and Forward links:

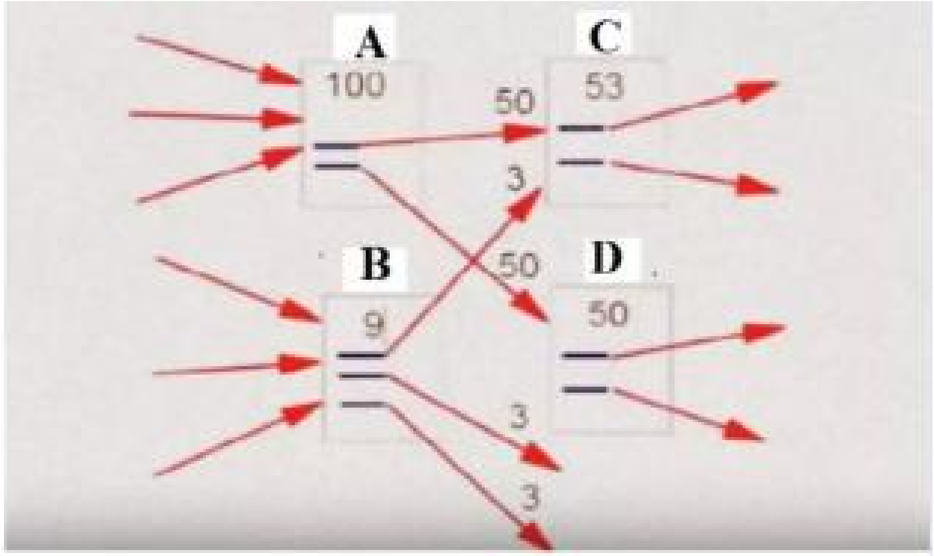
➢A and B are C's backlinks

➢C is A and B's forward link

Intuitively, a webpage is important if it has a lot of backlinks.

What if a webpage has only one link off www.yahoo.com?

- Initially 2 sites are there with PR of 100(Site A) and 9(Site B) respectively .
- Site A has two outlinks.So value of each outlink will be 100/2=50.
- Site B has three outlinks.So value of each outlink will be 9/3=3.

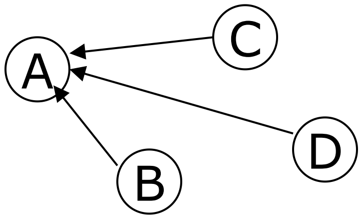



- Site C is pointed by one outlink of A (50) and one outlink of B (3) .So it has a value of (50+3)=53.
- Site D is pointed by one outlink of A(50) only .So it has a PR value of 50.
- Again C(53) has 2 outlinks so each outlink have value (53/2).While D(50) has two outlinks each with value of (50/2)=25.

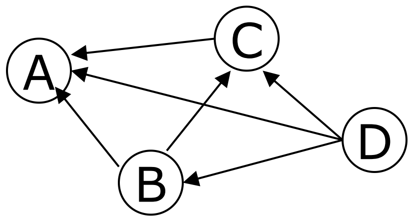
- So the order is **A > C > D > B**

# Simplified PageRank algorithm

- Assume four web pages: **A**, **B**,**C** and **D**. Let each page would begin with an estimated PageRank of 0.25.




$$PR(A) = PR(B) + PR(C) + PR(D).$$




$$PR(A) = \frac{PR(B)}{2} + \frac{PR(C)}{1} + \frac{PR(D)}{3}.$$

- L(A) is defined as the number of links going out of page A. The PageRank of a page A is given as follows:

$$PR(A) = \frac{PR(B)}{L(B)} + \frac{PR(C)}{L(C)} + \frac{PR(D)}{L(D)}.$$

# PageRank algorithm including damping factor

- Assume page A has pages B, C, D ..., which point to it. The parameter d is a damping factor which can be set between 0 and 1. Usually set d to 0.85. The PageRank of a page A is given as follows:

$$PR(A) = 1 - d + d\left(\frac{PR(B)}{L(B)} + \frac{PR(C)}{L(C)} + \frac{PR(D)}{L(D)} + \cdots\right)$$

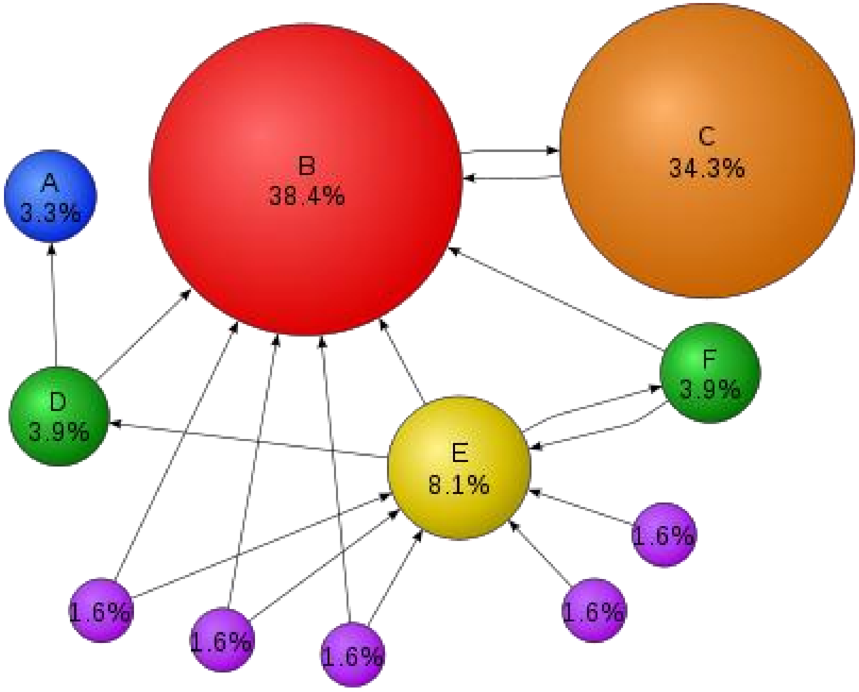
# Intuitive Justification

- A "random surfer" who is given a web page at random and keeps clicking on links, never hitting "back“, but eventually gets bored and starts on another random page.
    - The probability that the random surfer visits a page is its PageRank.
    - The d damping factor is the probability at each page the "random surfer" will get bored and request another random page.
- A page can have a high PageRank
    - If there are many pages that point to it
    - Or if there are some pages that point to it, and have a high PageRank.

# PageRank's Surfer Model

1. Follow edges uniformly with probability $\alpha$, and
2. Randomly jump to any page with probability 1-$\alpha$—we'll assume everywhere is equally likely

Using this Surfer Model for a given Web structure PageRank obtains a probability distribution; for example:




"The places we find the surfer most often are important pages"

# PageRank Computation

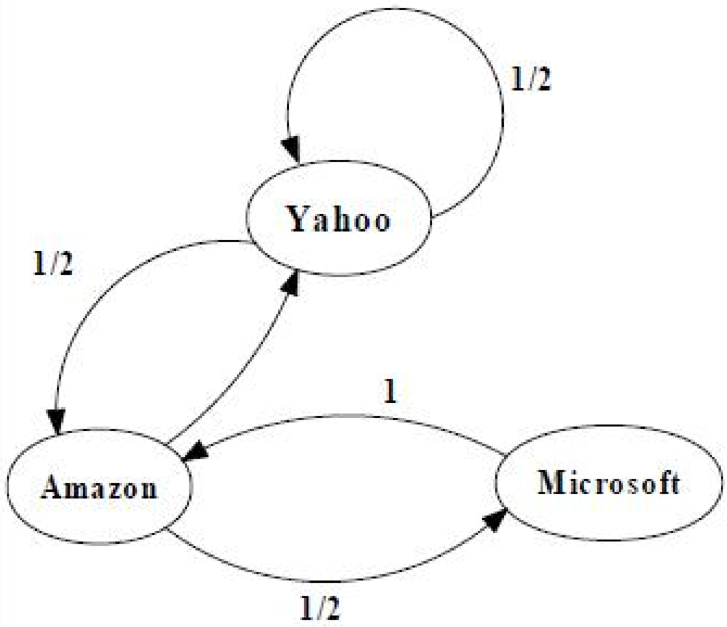
- Target
  - Solve the steady-state probability vector π, which is the PageRank of the corresponding Web page.
  - $\pi P = \lambda\ \pi$, $\lambda$ is 1 for stochastic matrix.
- Method
  - Power iteration.
  - Given an initial probability distribution vector x0
  - x0*P=x1, x1*P=x2 … Until the probability distribution converges. (Variation in the computed values are below some predetermined threshold.)

# A Simple Version of PageRank

$$R(u) = c \sum_{v \in B_u} \frac{R(v)}{N_v}$$

- u: a web page
- $B_u$: the set of u's backlinks
- $N_v$: the number of forward links of page v
- c: the normalization factor to make $||R||_{L1} = 1$ ($||R||_{L1} = |R_1 + \ldots + R_n|$)

# An example of Simplified PageRank



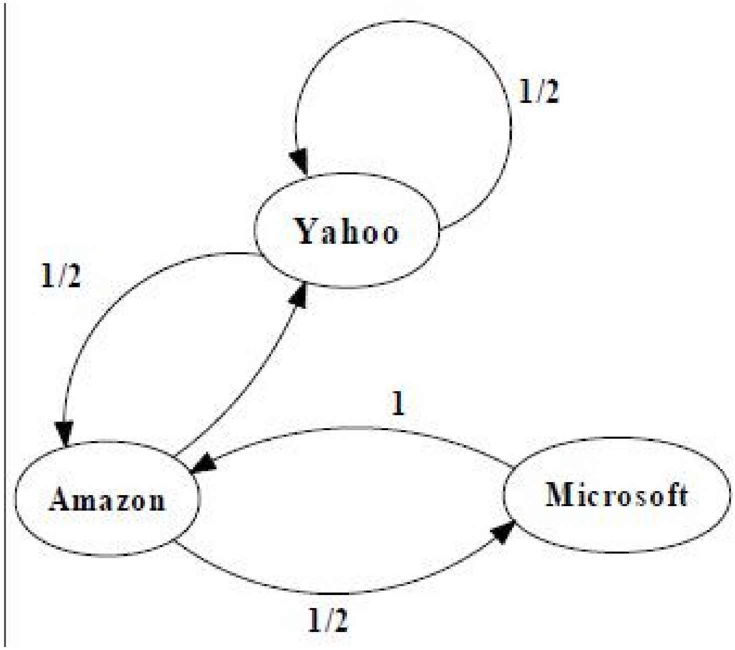

$$M = \begin{bmatrix} 1/2 & 1/2 & 0 \\ 1/2 & 0 & 1 \\ 0 & 1/2 & 0 \end{bmatrix}$$

$$\begin{bmatrix} \text{yahoo} \\ \text{Amazon} \\ \text{Microsoft} \end{bmatrix} = \begin{bmatrix} 1/3 \\ 1/3 \\ 1/3 \end{bmatrix}$$

$$\begin{bmatrix} 1/3 \\ 1/2 \\ 1/6 \end{bmatrix} = \begin{bmatrix} 1/2 & 1/2 & 0 \\ 1/2 & 0 & 1 \\ 0 & 1/2 & 0 \end{bmatrix} \begin{bmatrix} 1/3 \\ 1/3 \\ 1/3 \end{bmatrix}$$

PageRank Calculation: first iteration

# An example of Simplified PageRank



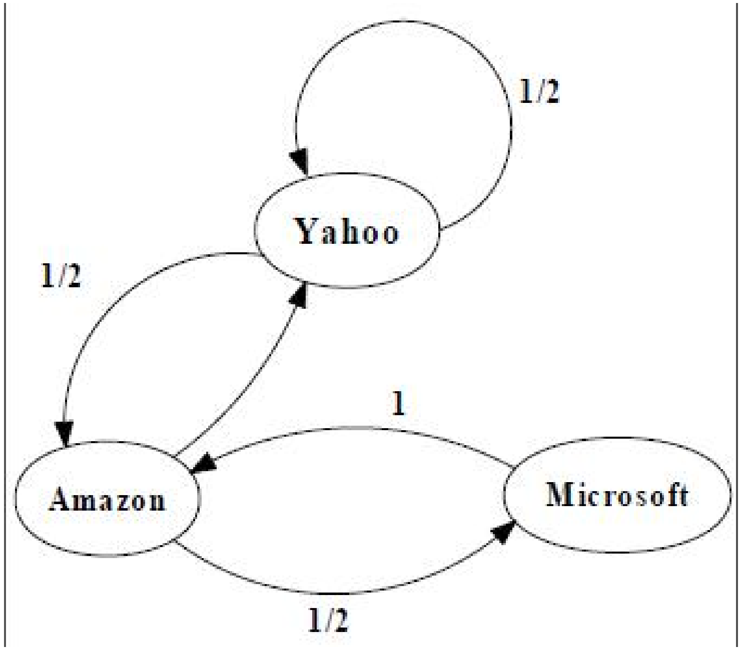

$$M = \begin{bmatrix} 1/2 & 1/2 & 0 \\ 1/2 & 0 & 1 \\ 0 & 1/2 & 0 \end{bmatrix}$$

$$\begin{bmatrix} \text{yahoo} \\ \text{Amazon} \\ \text{Microsoft} \end{bmatrix} = \begin{bmatrix} 1/3 \\ 1/3 \\ 1/3 \end{bmatrix}$$

$$\begin{bmatrix} 5/12 \\ 1/3 \\ 1/4 \end{bmatrix} = \begin{bmatrix} 1/2 & 1/2 & 0 \\ 1/2 & 0 & 1 \\ 0 & 1/2 & 0 \end{bmatrix} \begin{bmatrix} 1/3 \\ 1/2 \\ 1/6 \end{bmatrix}$$

PageRank Calculation: second iteration

# An example of Simplified PageRank




$$M = \begin{bmatrix} 1/2 & 1/2 & 0 \\ 1/2 & 0 & 1 \\ 0 & 1/2 & 0 \end{bmatrix}$$

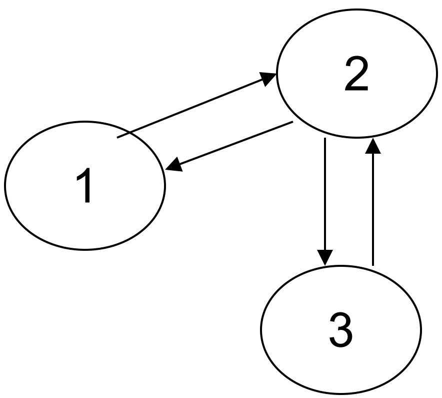
$$\begin{bmatrix} \text{yahoo} \\ \text{Amazon} \\ \text{Microsoft} \end{bmatrix} = \begin{bmatrix} 1/3 \\ 1/3 \\ 1/3 \end{bmatrix}$$

$$\begin{bmatrix} 3/8 \\ 11/24 \\ 1/6 \end{bmatrix} \begin{bmatrix} 5/12 \\ 17/48 \\ 11/48 \end{bmatrix} \ldots \begin{bmatrix} 2/5 \\ 2/5 \\ 1/5 \end{bmatrix}$$

Convergence after some iterations

# Exercise on PageRank

- Consider a Web graph with three nodes 1, 2, and 3. The links are as follows: 1->2, 3->2, 2->1, 2->3. Write down the transition probability matrices P for the surfer’s walk with teleporting, with the value of teleport probability α=0.5.




$$A=\begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

Each 1 divied by the number of ones in this row

$$(1-\alpha)*\begin{bmatrix} 0 & 1 & 0 \\ ½ & 0 & ½ \\ 0 & 1 & 0 \end{bmatrix}$$

$$+$$

$$\alpha*\begin{bmatrix} 1/3 & 1/3 & 1/3 \\ 1/3 & 1/3 & 1/3 \\ 1/3 & 1/3 & 1/3 \end{bmatrix} = \begin{bmatrix} 1/6 & 2/3 & 1/6 \\ 5/12 & 1/6 & 5/12 \\ 1/6 & 2/3 & 1/6 \end{bmatrix}$$

# Dangling Links

- Links that point to any page with no outgoing links
- Most are pages that have not been downloaded yet
- Affect the model since it is not clear where their weight should be distributed
- Do not affect the ranking of any other page directly
- Can be simply removed before pagerank calculation and added back afterwards

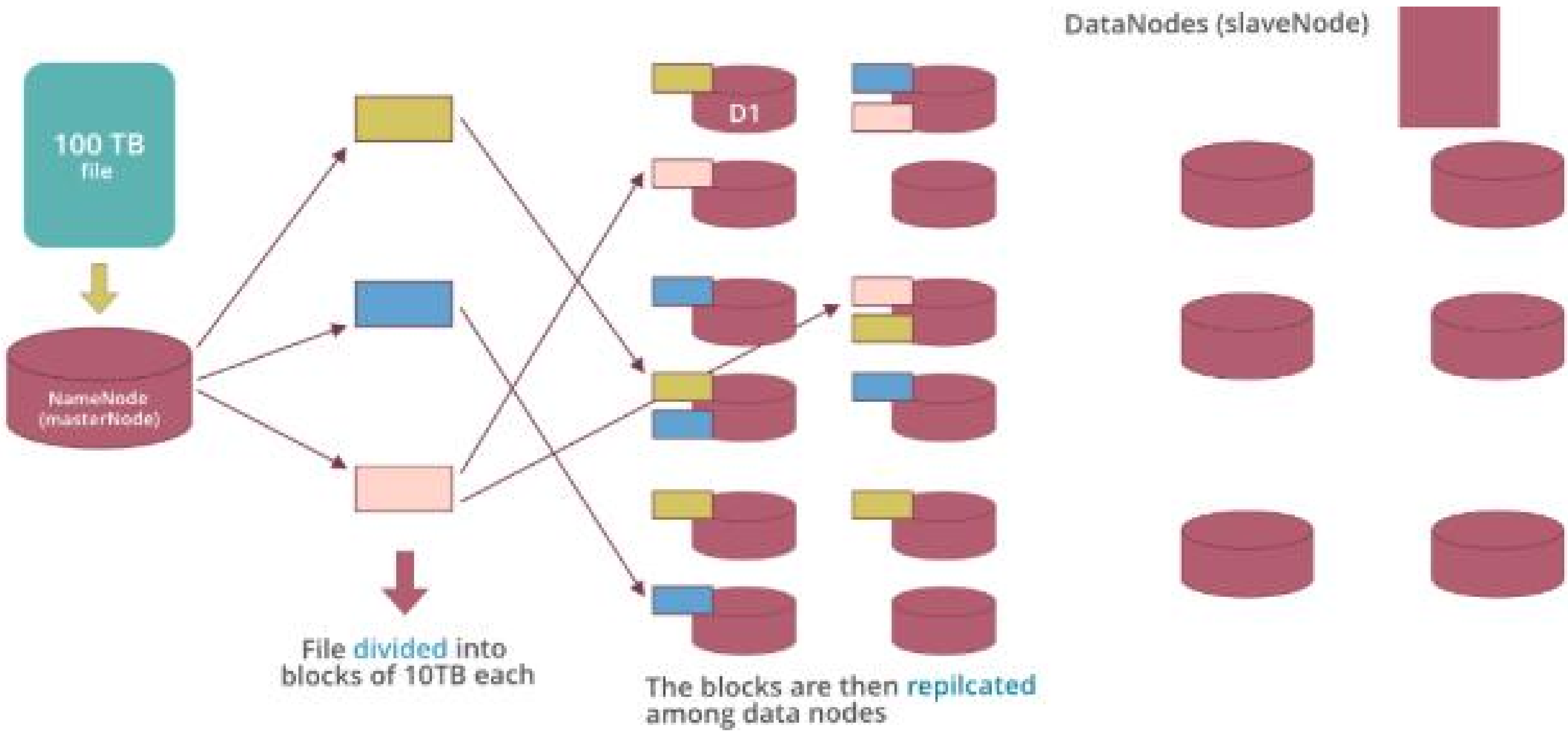
# Google bomb

- Because of the PageRank, a page will be ranked higher if the sites that link to that page use consistent anchor text.

- A Google bomb is created if a large number of sites link to the page in this manner.

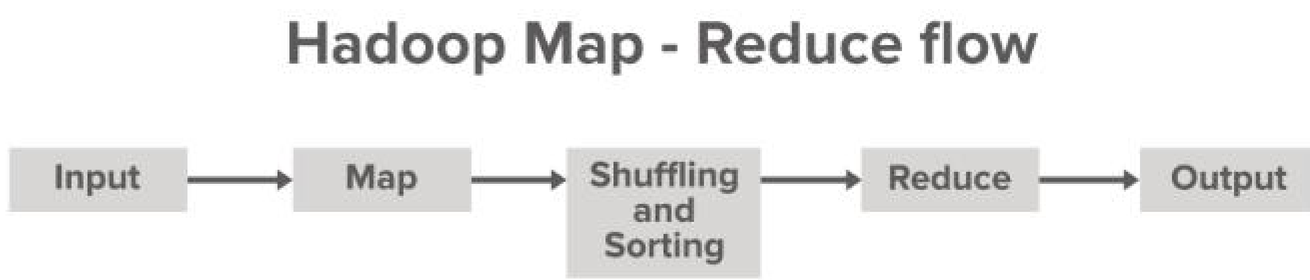
- search term "more evil than Satan himself" → the Microsoft homepage as the top result.

# HDFS


DataNodes (slaveNode)
100 TB
file
NameNode
(masterNode)
D1
File divided into
blocks of 10TB each
The blocks are then repilcated
among data nodes

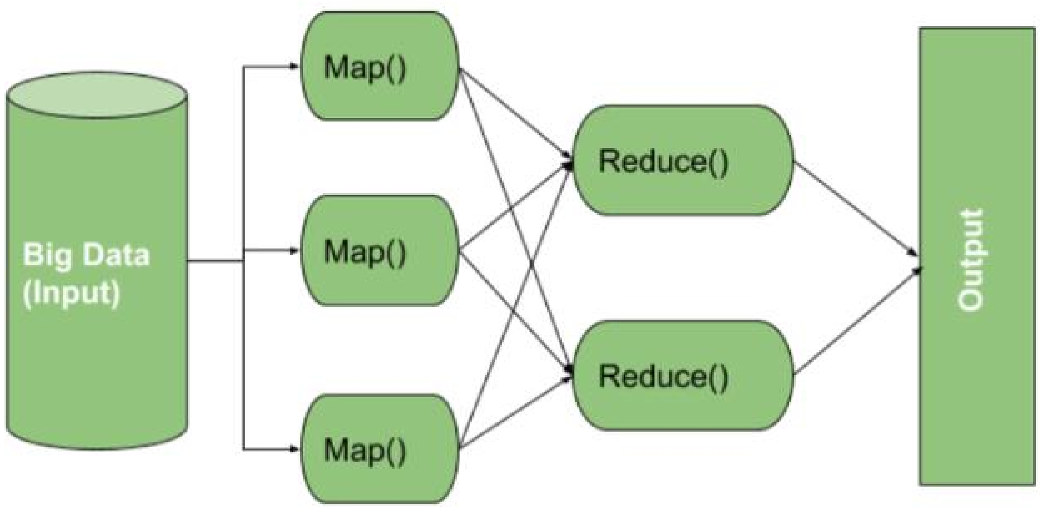
# Hadoop-MapReduce

- Hadoop MapReduce enables us to process massive graph datasets efficiently.”



# Key components of MapReduce

- The MapReduce consists of two primary phases such as the map phase and the reduces phase. Each phase contains the key-value pairs as its input and output and it also has the map function and reducer function within it.



# How MapReduce Works

- **1.Input Split**
  - The input data (e.g., a big log file or dataset) is divided into smaller chunks called Input Splits. Each split is processed independently by a separate Mapper.
  - **Example:** If you have a 1 GB file and Hadoop splits it into four 256 MB chunks, it will use 4 Mappers one for each chunk.
- **2. Mapper Phase**
  - Each Mapper runs in parallel on different nodes and processes one input split.
  - What it does:
  - Reads the input data line by line
  - Transforms each line into key-value pairs
  - Stores the intermediate output locally (not yet in HDFS)
  - **Example:** If the task is counting words, the Mapper reads: "Data is power"--> emits:
- *("Data", 1), ("is", 1), ("power", 1)*

- **3. Shuffling & Sorting**
  - This is a behind-the-scenes phase handled by Hadoop after mapping is done.
  - What it does:
    - Shuffles intermediate key-value pairs across the cluster
    - Groups all values with the same key
    - Sorts them by key before sending to the Reducer
  - **Example:** From all Mappers, these pairs:
  - *("Data", 1), ("Data", 1), ("power", 1)*are grouped into:*("Data", [1, 1]), ("power", [1])*
- **4. Reducer Phase**
  - Each Reducer receives a list of values for each unique key.
  - What it does:
    - Applies aggregation logic (e.g., sum, average, filter)
    - Generates the final key-value output
    - Stores the result in HDFS
  - **Example:** For word count, the Reducer gets:
  - *("Data", [1, 1]) --> outputs: ("Data", 2)*
  - The final output is saved in files like: part-r-00000

# Data Science Ethics

Week # 15
Course Instructor: Mahrukh Khan

# Ethics in Data Science

- Data science now influences:
  - Healthcare (diagnosis models)
  - Finance (credit scoring)
  - Government (predictive policing, welfare fraud detection)
  - Employment (screening CVs)
  - Social media (recommendation systems)

- In September 2018, hackers injected malicious code into British Airways' website, diverting traffic to a fraudulent replica site. Customers then unknowingly gave their information to fraudsters, including login details, payment card information, address, and travel booking information.
- In 2019, after Apple introduced its credit card to consumers, allegations of an algorithm with gender bias emerged. Several prominent tech executives (including Steve Wozniak, the famous technologist and cofounder of Apple) described receiving exponentially higher credit limits than their wives, with whom they shared assets. Besides gender, no clear factors could suggest such a difference.
- In March 2021, the privacy of over 533 million Facebook users was compromised when their data was posted on an open hackers' forum. It was one of the largest data breaches of all time. The incident raised concerns about how organizations store and secure personal information and whether they should be allowed access to such data in the first place.

# Principles of Data Ethics

- Transparency: Clear communication about data collection, storage, and sharing practices is vital. Users should understand how their information is being used, without needing legal expertise to navigate terms and conditions.
- Accountability: Organizations must take responsibility for the data they collect, including protecting it from breaches and misuse. This principle is crucial for maintaining trust between companies and their users.
- Individual Agency: Individuals should have control over their personal data, including the ability to access, correct, or delete their information. Ensuring this agency is a fundamental human right in the digital age.
- Data Privacy: Privacy is the expectation that personal data will be protected from unauthorized exposure. Organizations must uphold this expectation by implementing robust security measures and honoring the agreements made with users.

# Ethical Frameworks

- Consequentialism – focus on outcomes
- Deontology – focus on rules and rights
- Virtue Ethics – focus on moral character
- Responsible AI Principles (Common Industry themes):
  - Fairness
  - Accountability
  - Transparency
  - Privacy & Safety

# Bias in Machine Learning Models

- Bias is systematic error that leads to unfair outcomes for certain groups.
- Data bias(Sampling Bias, Measurement Bias, Historical Bias,Labeling Bias)
- Algorithmic bias->The model learns harmful correlations
- Societal/Deployment  bias->When a system is used in a harmful context

# How bias enters an ML pipeline?

- If a bank uses historical loan approval data where fewer women received loans, what happens when an ML model is trained on it?

# Data Privacy & Surveillance

- The right of individuals to control how their data is collected, used, stored, and shared.
- Personally Identifiable Information (PII)
- Anonymization vs. De-identification vs. Re-identification
- Informed consent
- Data minimization (collect only what is needed)
- Purpose limitationSurveillance Technologies
  - CCTV + facial recognition
  - Phone metadata tracking
  - GPS, IoT sensors
  - Social media monitoring
  - Predictive policing tools

# Government Norms for Collecting and Storing Data under Legal and Regulatory Frameworks

- Data Protection Laws and Security Standards:
- Data Retention Policies:
- Compliance and Audits
- International Data Transfer Regulations

# Algorithmic Fairness

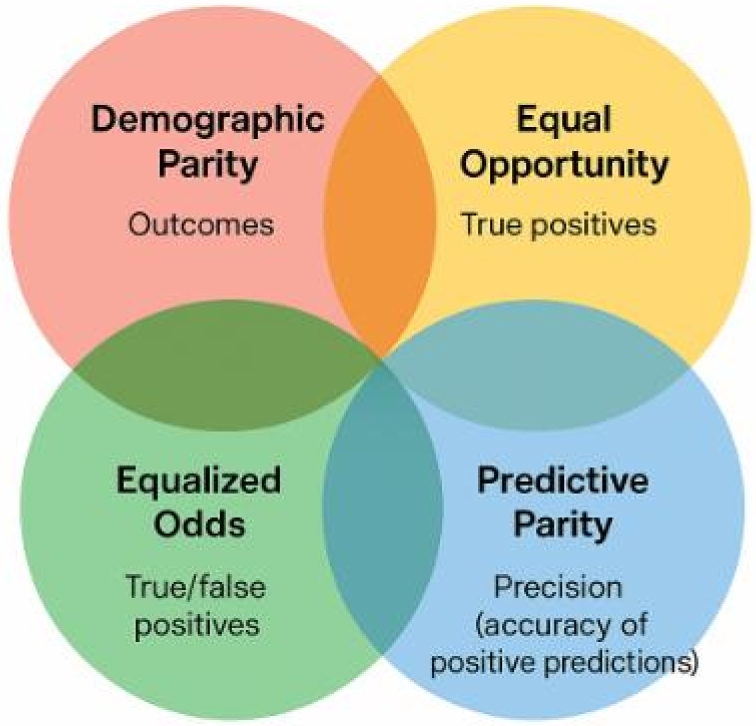
- Ensuring ML systems do not discriminate against individuals or groups.
- How?

| Pre-processing Methods | In-processing Methods | Post-processing Methods |
|---|---|---|
| Re-sampling (oversampling minorities)<br><br>Data balancing<br><br>Removing sensitive attributes | Fairness constraints in loss function<br><br>e.g., Demographic parity<br><br>Equal opportunity | Adjusting outputs<br><br>Thresholding by group<br><br>Recalibration |

# Fairness Metrics

- Demographic Parity
  - Prediction should be independent of sensitive attribute.
- Equal Opportunity
  - Equal true positive rates across groups.
- Equalized Odds
  - Equal TPR and FPR across groups.
- Predictive Parity
  - Equal precision across groups.

- Tradeoffs->Impossible to satisfy all fairness metrics at once



- If a hiring algorithm favors men because they historically applied more often for software engineering jobs— which fairness metric is appropriate?

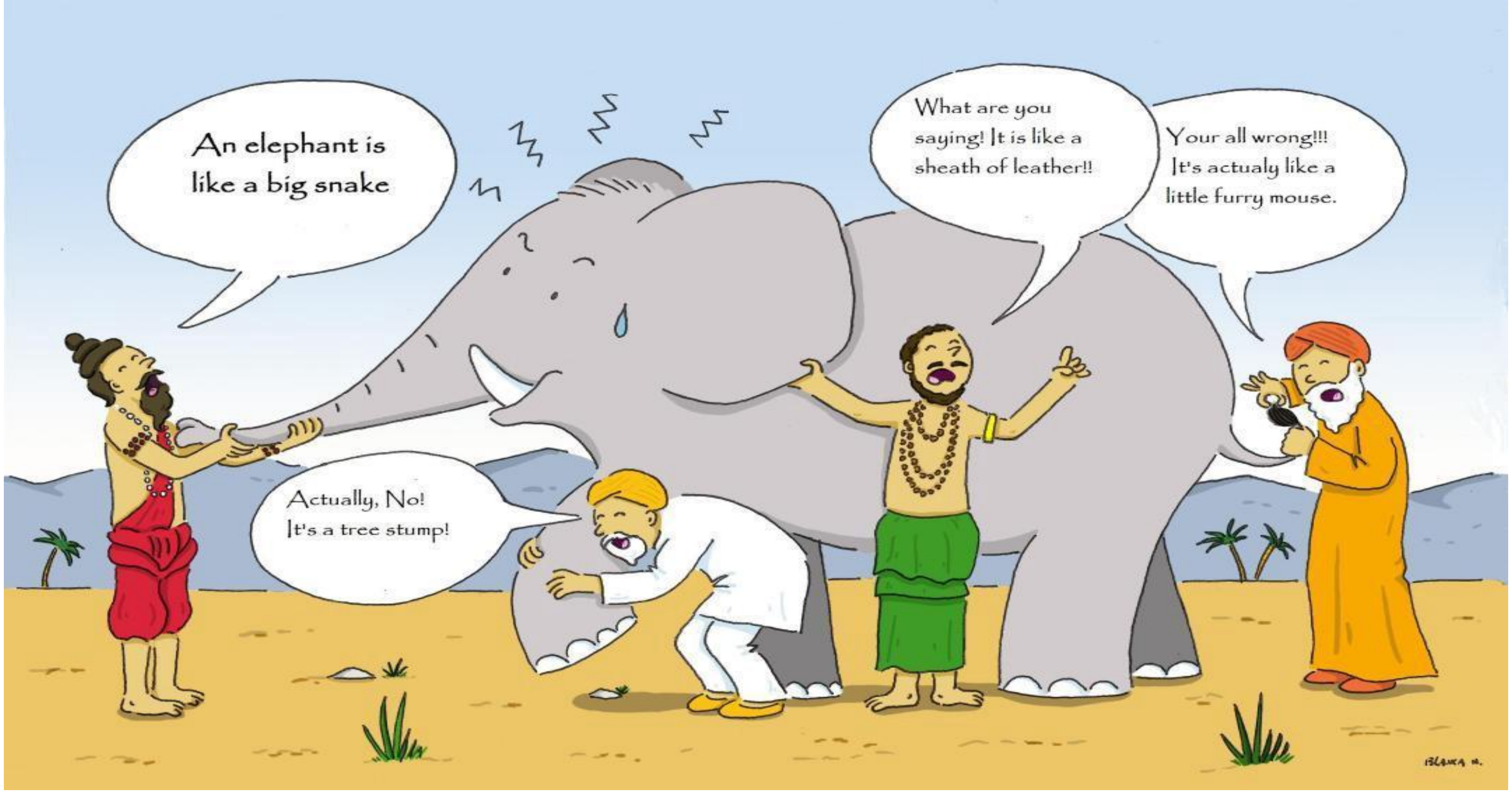
# Debate Activity

- Topic: Should governments use facial recognition for public safety?
- - Group A: Support
- - Group B: Oppose

# Ensemble Learning

Week # 15
Course Instructor: Mahrukh Khan

# Overview


An elephant is like a big snake
Actually, No! It's a tree stump!
What are you saying! It is like a sheath of leather!!
Your all wrong!!! It's actualy like a little furry mouse.

# Goal of Supervised Learning?

- Minimize the probability of model prediction errors on ***future*** data

- Two Competing Methodologies
  - Build **one** really good model
    - Traditional approach
  - Build **many** models and average the results
    - Ensemble learning (more recent)

# The Single Model Philosophy

- Motivation: Occam's Razor
  - "one should not increase, beyond what is necessary, the number of entities required to explain anything"
- Infinitely many models can explain any given dataset
  - Might as well pick the smallest one…

# What does *Ensemble* mean?

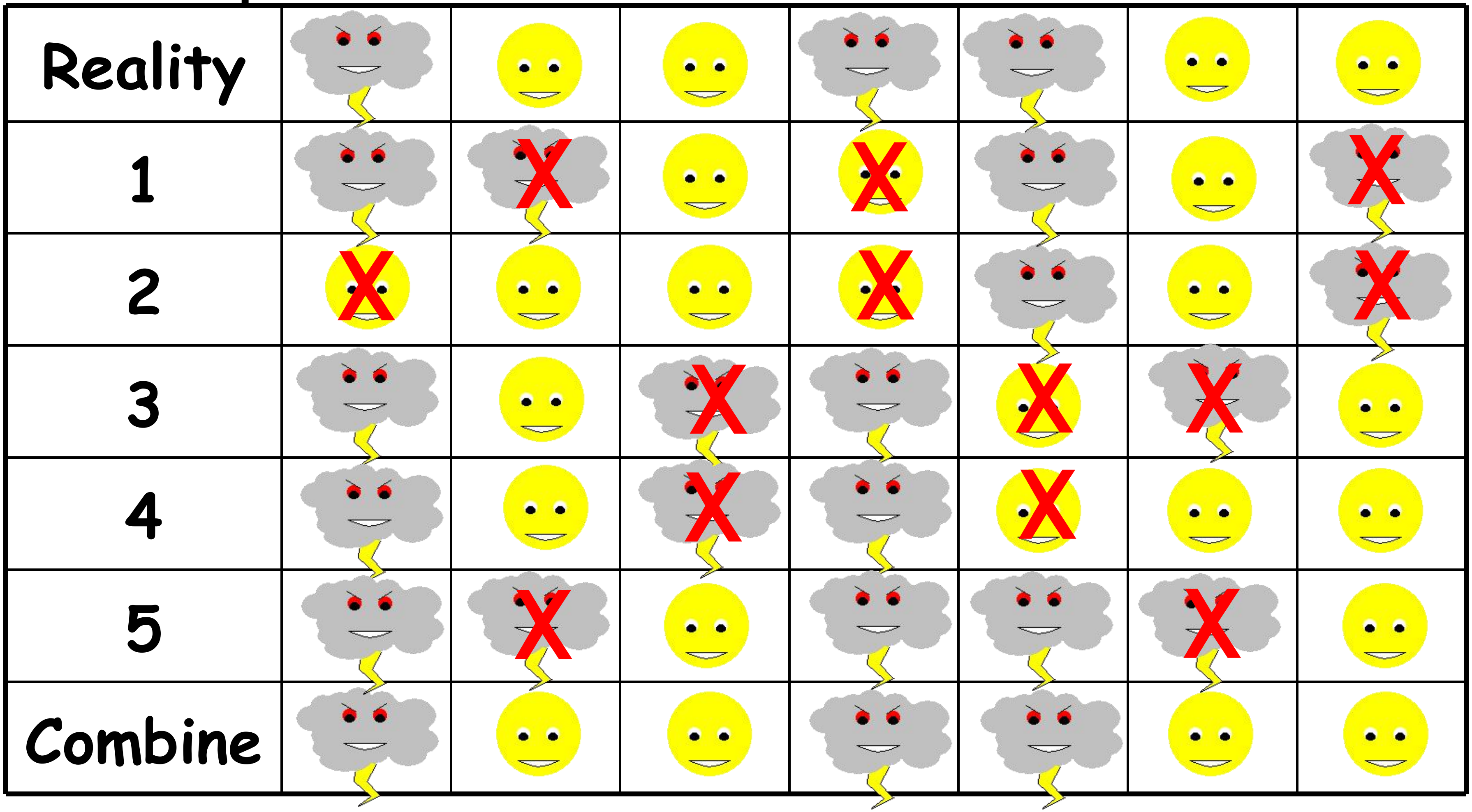
- A group of items, viewed as a whole rather than individually.
- In AI, a group of learners combine to form a better learner.
- Only through averaging do we get at the truth!
- It's too hard (*impossible*?) to build a single model that works best
- Two types of approaches:
  - Models that don't use randomness
  - Models that incorporate randomness

# What is Ensemble learning ?

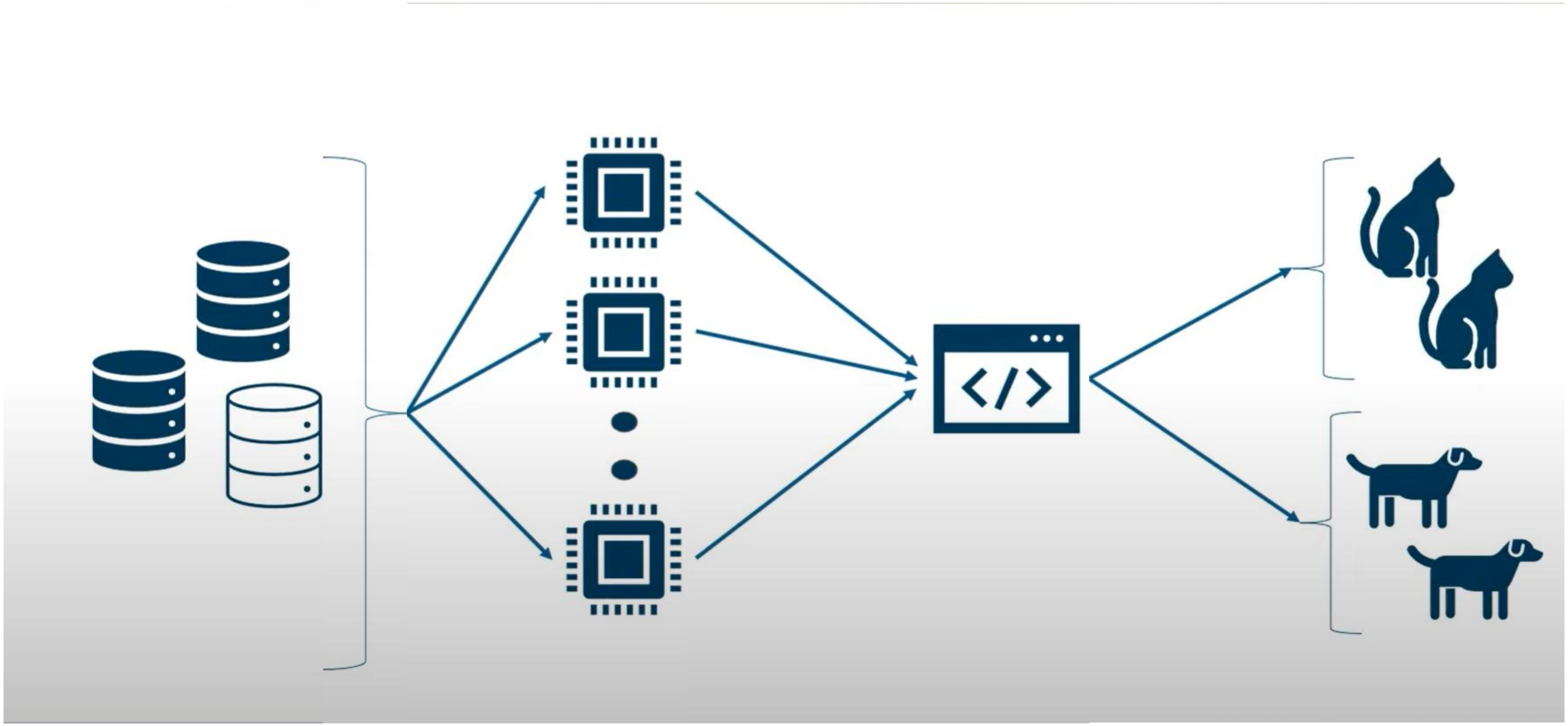
- Ensemble learning is a machine learning paradigm where multiple learners are trained to solve the same problem.

- Ensemble learning take the attitude that more predictors can be better than any single one and by learning many we can do a better job.

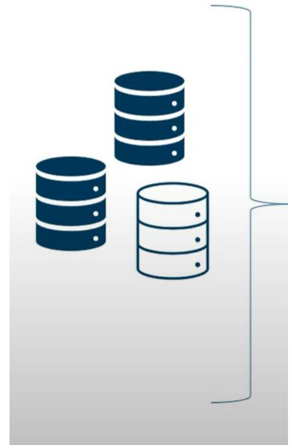
# Example: Weather Forecast



# Ensemble of classifiers:

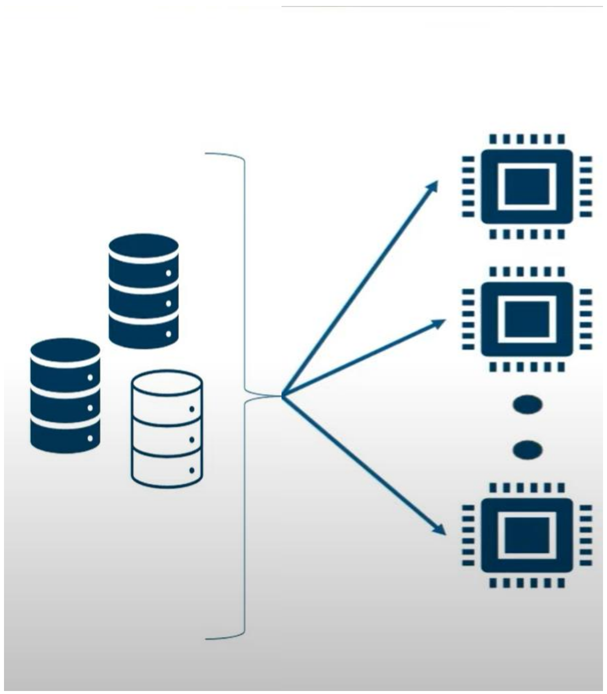


- We build a collection of classifiers, say $h_1$, $h_2$, $h_3$, ….. $h_n$
  And then we construct a final classifier by combining their individual decisions.



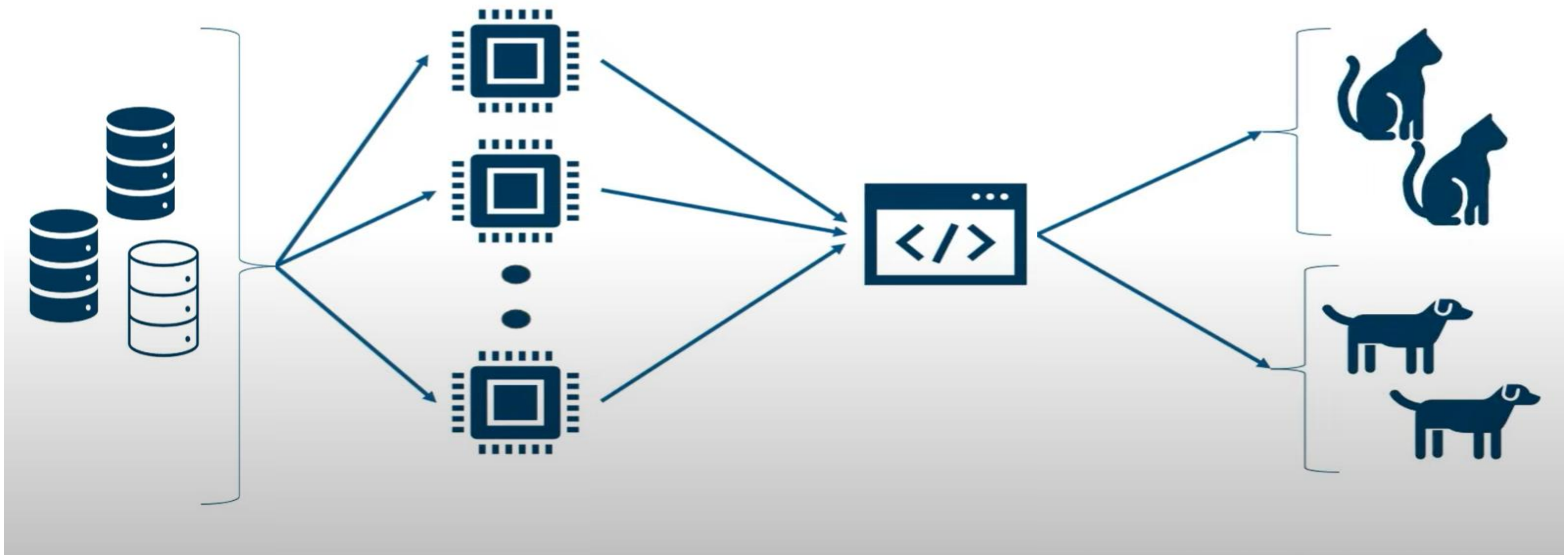
Random Data Samples

Random sampling is a part of the sampling techniques in which each sample has an equal probability of being chosen. A sample chosen randomly means, an unbiased representation of the total dataset



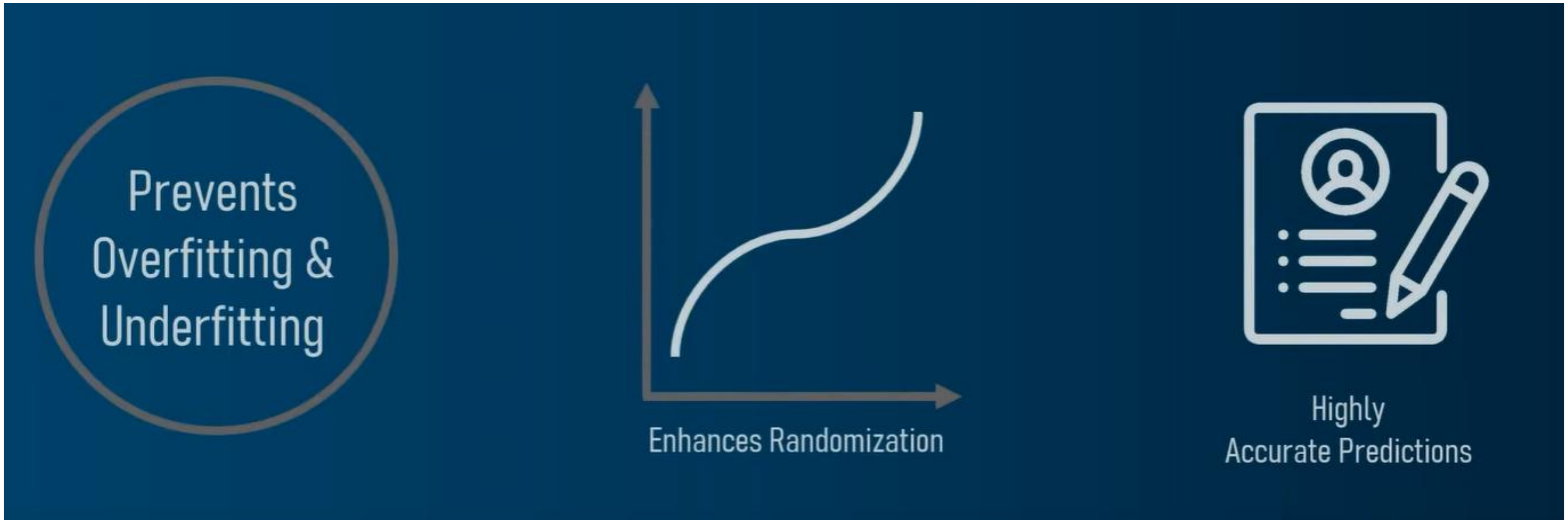
Weak Learners

A ‘weak’ learner is just one which performs relatively poor– its accuracy is above chance, but just barely. There is often, but not always, the added implication that it is computationally simple. Weak learner also suggests that many instances of the algorithm are being pooled together into to create a “strong” ensemble classifier.



**Final Model**

In machine learning, ensemble methods use multiple learning algorithms to obtain better predictive performance than could be obtained from any of the learning algorithms alone.




**Need for Ensemble Learning**

**Ensembles are predictive models that combine predictions from two or more other models. Ensemble learning method are popular and the go-to technique when the best performance on a predictive modeling project is the most important outcome.**

# So Why Does This Work?

- “No Free Lunch"Theorem:No single algorithm wins all the time!
- Wisdom of the Crowd: When combining multiple independent decisions, each of which is at least more accurate than random guessing,random  errors cancel each other out and correct decisions are reinforced.
- Human ensembles are demonstrably better.
    - How many jellybeans are in the jar?
    - Who Wants to Be a Millionaire: “Ask the audience"

# Core of Ensemble Learning

- Combining weak classifiers (of the same type) in order to produce a strong classifier
- Condition: diversity among the weak classifiers (No gain if all the classifier make the same mistake).
- Each classifier is “weak" but the ensemble is “strong."

# Why Ensemble Learning Improves Accuracy

Assume that classification errors are independent.

Let $p$ = probability that a classifier makes an error.

- The probability that $k$ of the $n$ classifiers make an error is:

$$\sum_{k=1}^{n} \binom{n}{k} p^k (1-p)^{n-k}$$

- If $p < 1/2$, the probability that more than half of the classifiers are in error is:

$$P\left(k > \frac{n}{2}\right) = P\left(N(0,1) > \frac{n/2 - np}{\sqrt{np(1-p)}}\right)$$

$$= P\left(N(0,1) > \frac{\frac{1}{2} - p}{\sqrt{p(1-p)}} \cdot \sqrt{n}\right) \sim e^{-cn} \to 0,$$

$as\, n \to \infty.$

# Two Key Questions

1. How to get different learners?

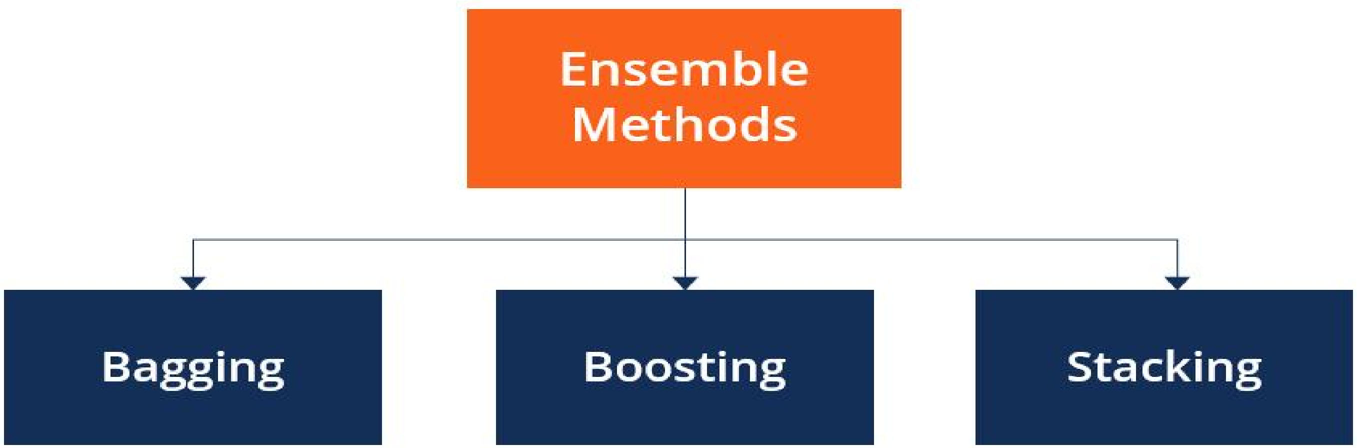
1. How to combine learners?

# Where Do Learners Come From

- Partitioning the data.

- Using
  - Different feature subsets.
  - Different algorithms.
  - Different parameters of the same algorithm.

# Combining learners



# What is Bagging?

- Bagging comes from a combination of words, Bootstrap + Aggregating.
- A machine learning ensemble meta-algorithm designed to improve the stability and accuracy of machine learning algorithms used in statistical classification and regression.
- It also reduces variance and helps to avoid overfitting.

# Bagging

- Main Assumption:
  - Combining many unstable predictors to produce a ensemble (stable) predictor.
  - Unstable Predictor: small changes in training data produce large changes in the model.
    - e.g. Neural Nets, trees
    - Stable: SVM (sometimes), Nearest Neighbor.
- Hypothesis Space
  - Variable size (nonparametric):
    - Can model any function if you use an appropriate predictor (e.g. trees)

# The Bagging Algorithm

Given data: $D = \{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_N, y_N)\}$

For $m = 1 : M$

- Obtain bootstrap sample from the training data
$D_m$
- Build a model from bootstrap data

$D$

$G_m(\mathbf{x})$ $D_m$

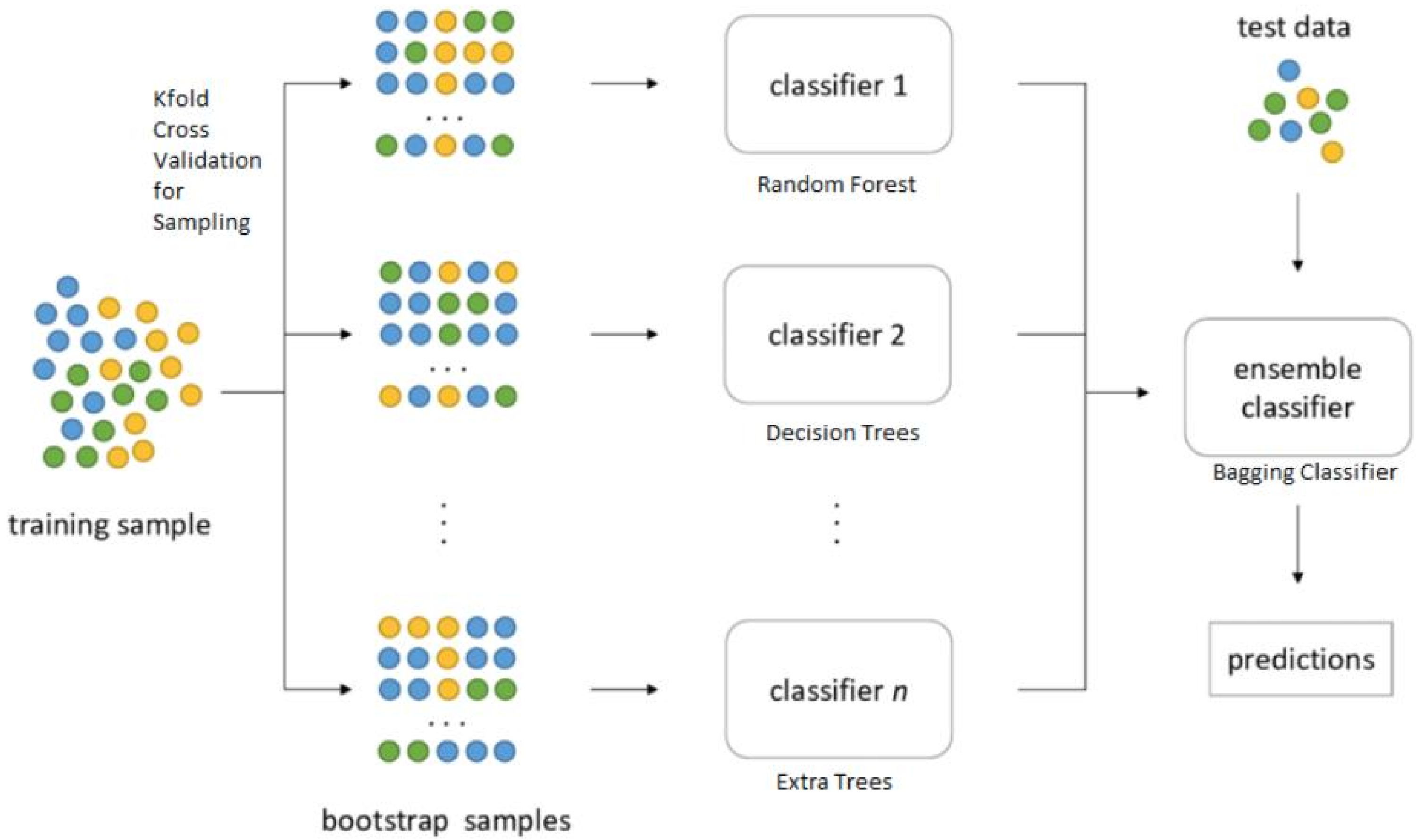
# The Bagging Model

- Regression

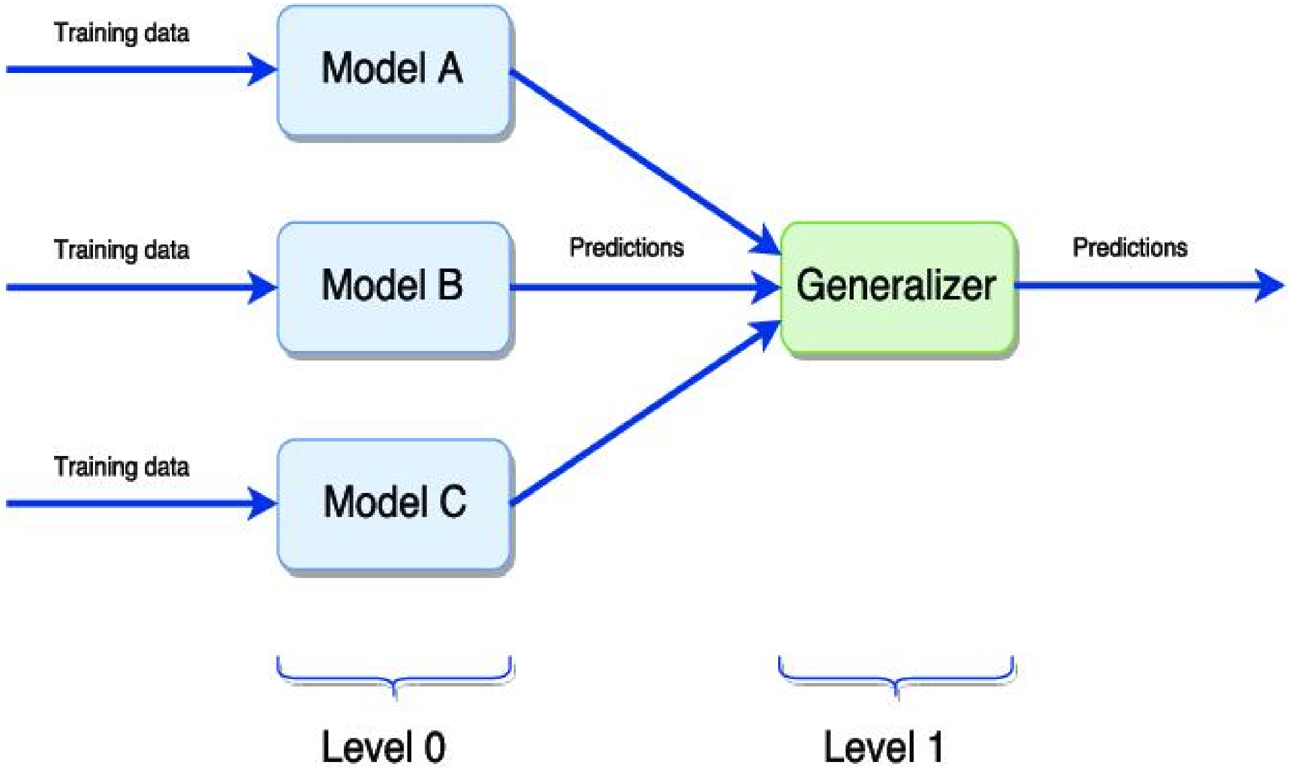
$$\hat{y} = \frac{1}{M} \sum_{m=1}^{M} G_m(\mathbf{x})$$

- Classification:
  - Vote over classifier outputs $G_1(\mathbf{x}), \ldots, G_M(\mathbf{x})$

# Bagging


Kfold
Cross
Validation
for
Sampling
training sample
bootstrap samples
classifier 1
Random Forest
classifier 2
Decision Trees
classifier n
Extra Trees
test data
ensemble
classifier
Bagging Classifier
predictions

# Stacking



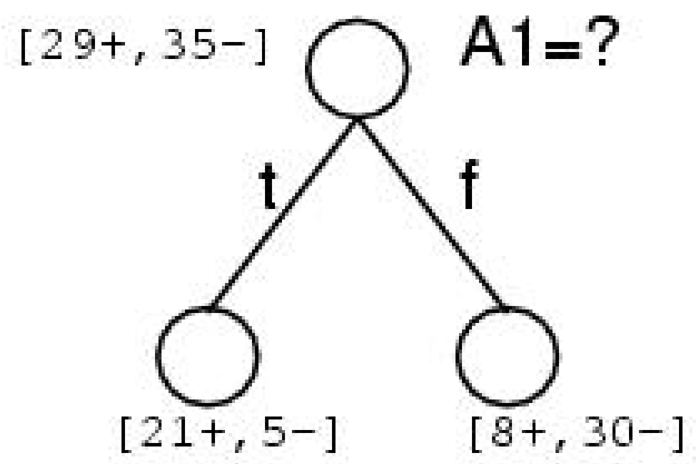
Training data
Training data
Training data
Model A
Model B
Model C
Predictions
Generalizer
Predictions
Level 0
Level 1

# Boosting

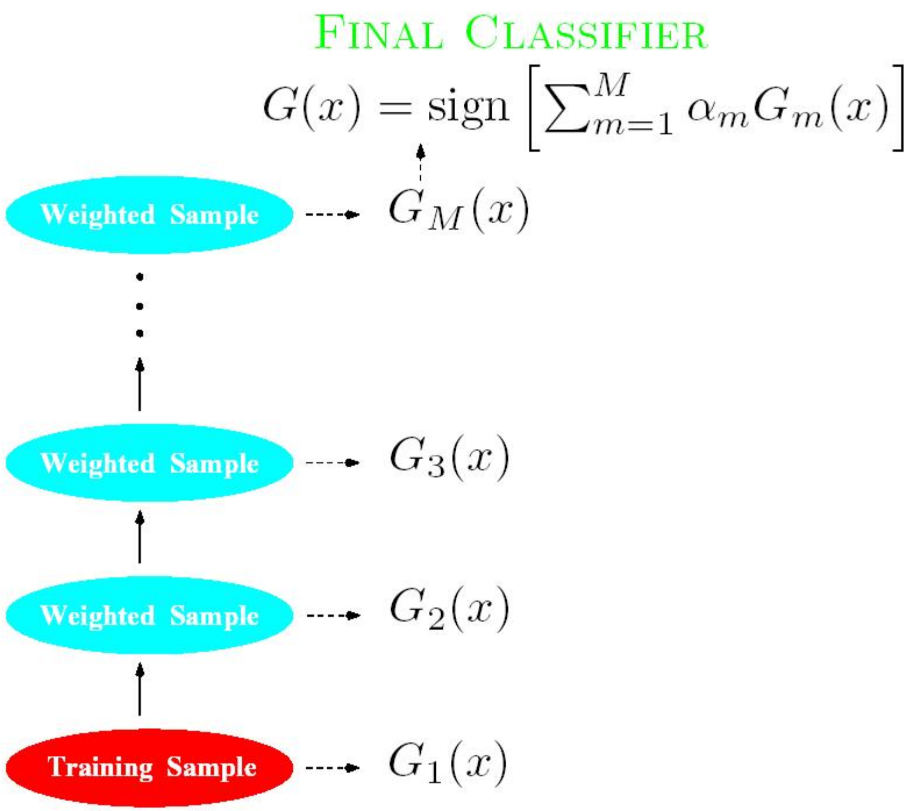
- Main Assumption:
  - Combining many weak predictors (e.g. tree stumps or 1-R predictors) to produce an ensemble predictor
  - The weak predictors or classifiers need to be **stable**
- Hypothesis Space
  - Variable size (nonparametric):
    - Can model any function if you use an appropriate predictor (e.g. trees)

# Commonly Used Weak Predictor (or classifier)

A Decision Tree Stump (1-R)



# Boosting




Each classifier $G_m(\mathbf{x})$ is trained from a weighted Sample of the training Data

# Boosting (Continued)

- Each predictor is created by using a biased sample of the training data
  - Instances (training examples) with high error are weighted higher than those with lower error
- Difficult instances get more attention
  - This is the motivation behind boosting

# Background Notation

- The $I(s)$ function is defined as:

$$I(s) = \begin{cases} 1 \text{ if } s \text{ is true} \\ 0 \quad \text{otherwise} \end{cases}$$

- The $\log(x)$ function is the natural logarithm

logarithm

# The AdaBoost Algorithm

(Freund and Schapire, 1996)

Given data: $D = \{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_N, y_N)\}$

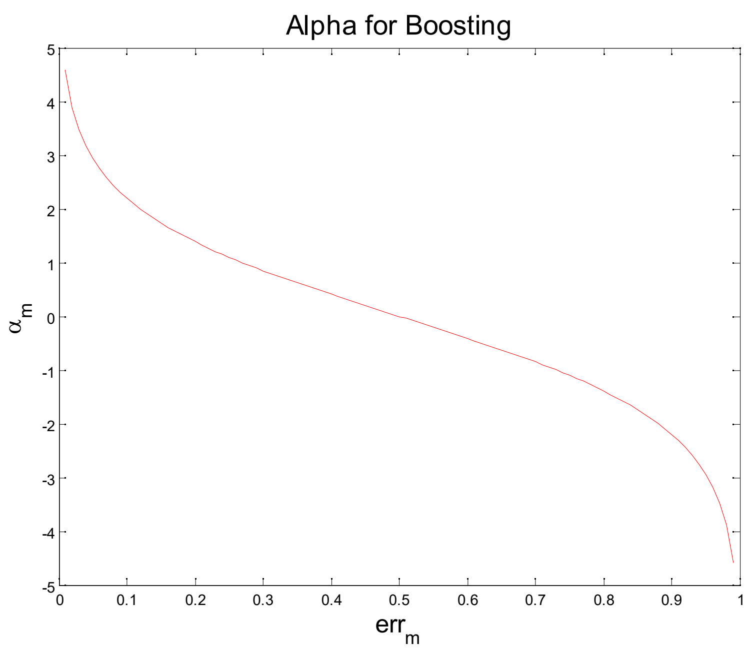
1. Initialize weights $w_i = 1/N, i = 1, \ldots, N$
2. For $m = 1 : M$
   a) Fit classifier $G_m(\mathbf{x}) \in \{-1, 1\}$ to data using weights $w_i$
   b) Compute
   $$err_m = \frac{\sum_{i=1}^{N} w_i I(y_i \neq G_m(\mathbf{x}_i))}{\sum_{i=1}^{N} w_i}$$
   c) Compute $\alpha_m = \log((1 - err_m)/err_m)$
   d) Set $w_i \leftarrow w_i \exp[\alpha_m I(y_i \neq G_m(\mathbf{x}_i))], \quad i = 1, \ldots, N$

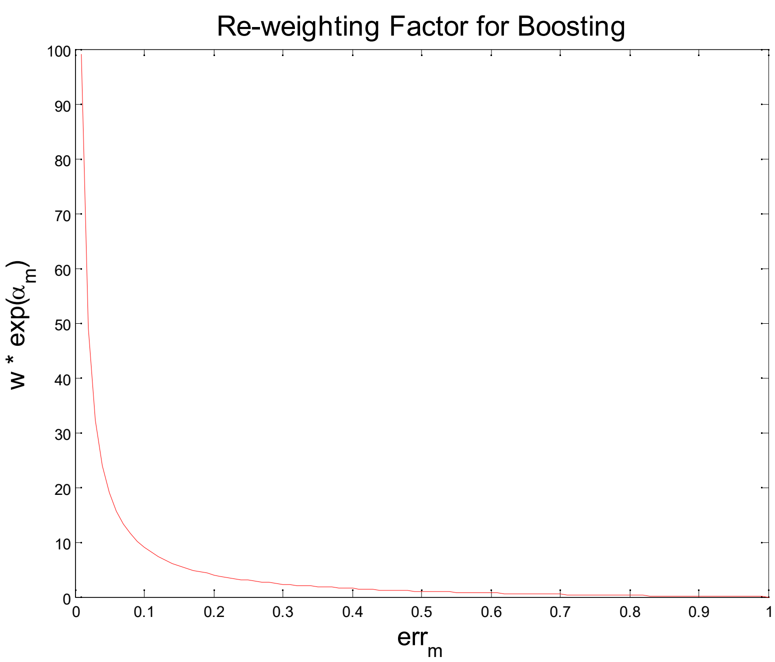
# The AdaBoost Model

$$\hat{y} = \operatorname{sgn}\left[\sum_{m=1}^{M} \alpha_m G_m(\mathbf{x})\right]$$

AdaBoost is NOT used for Regression!

# The Updates in Boosting

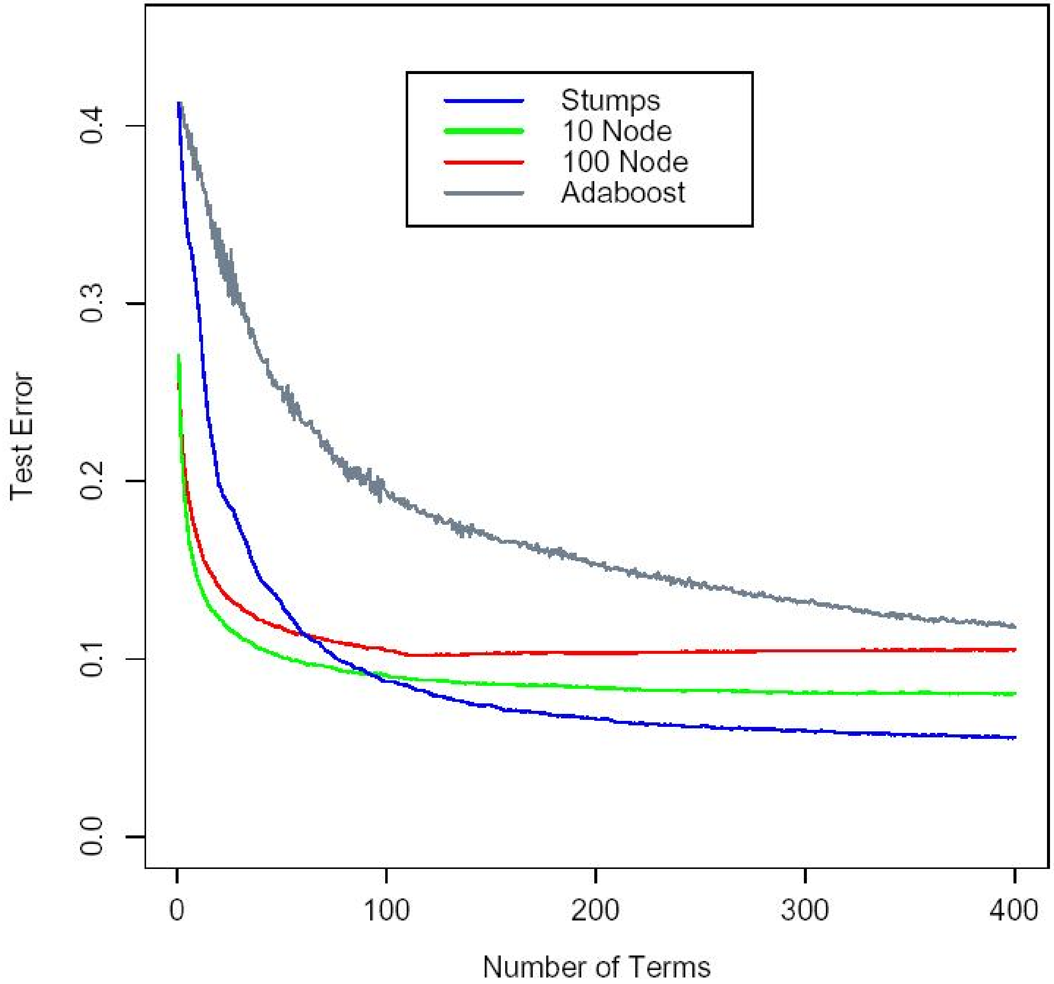


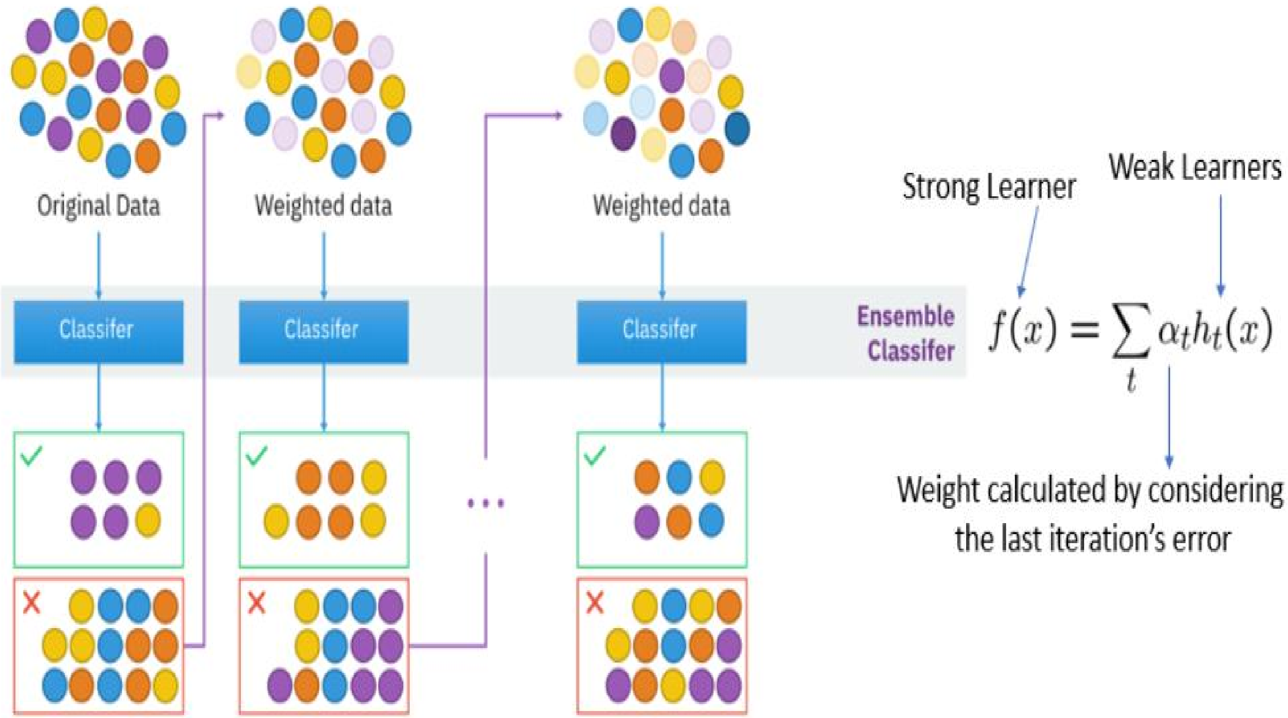
# Other Variations of Boosting

- Gradient Boosting
  - Can use any cost function
- Stochastic (Gradient) Boosting
  - Bootstrap Sample: Uniform random sampling (with replacement)
  - Often outperforms the non-random version

# Gradient Boosting



# Boosting


Original Data
Weighted data
Weighted data
Classifer
Classifer
Classifer
Ensemble Classifer
...
Strong Learner
Weak Learners
$f(x) = \sum_t \alpha_t h_t(x)$
Weight calculated by considering the last iteration's error

# Boosting

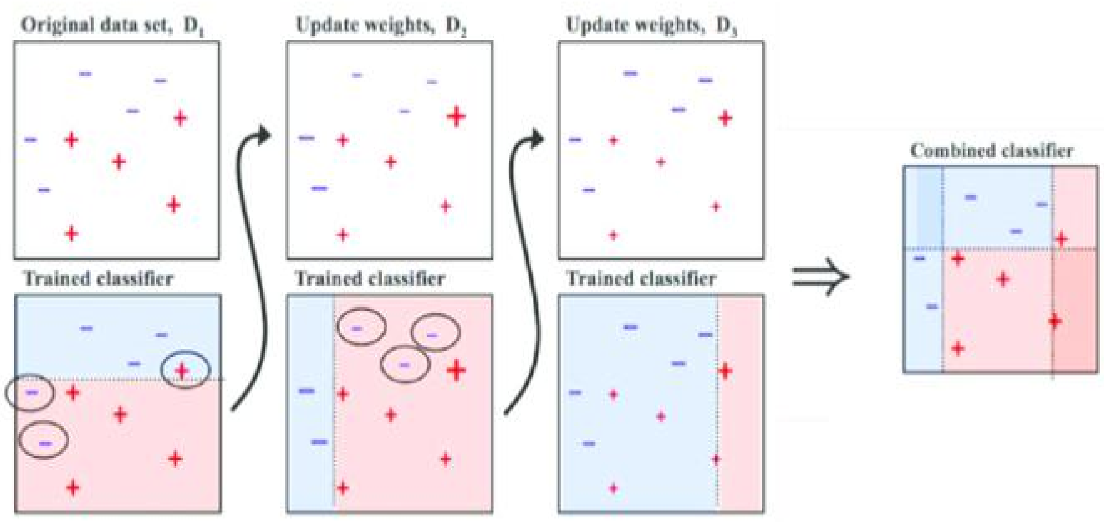
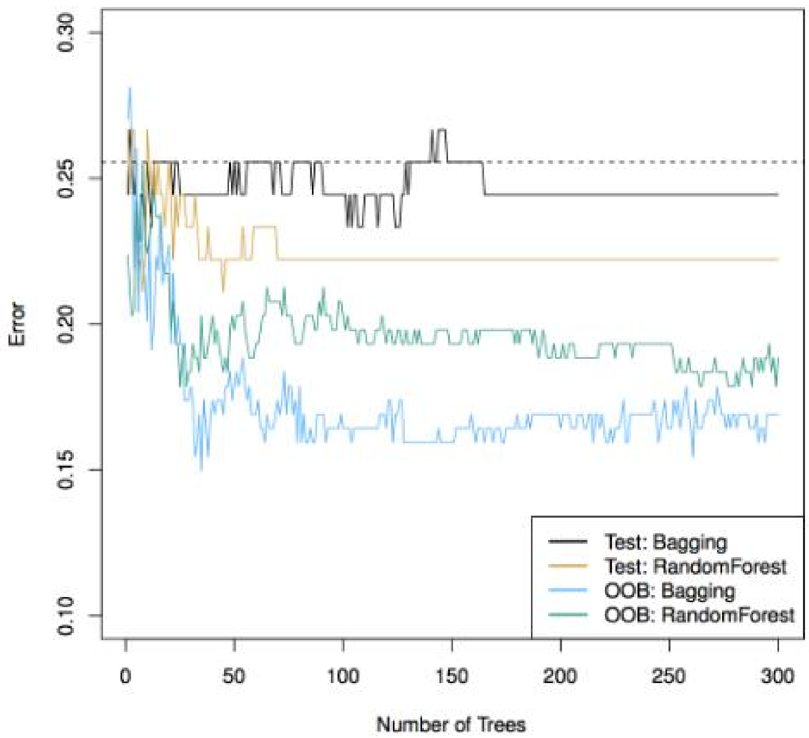



Image source https://www.pluralsight.com/guides/ensemble-methods-bagging-versus-boosting

# What is Random Forest?

- Forests are made up of trees and Random forests are made up of Decision Trees.
- Random Forest is an ensemble learning method for classification, regression that operates by creating a multitude of decision trees.
- A general method random forest was first proposed by Ho in 1995.

# Bagging VS Random Forest




- Random Forest are among the best classifiers in the world.
- The 3 tuning parameters:
  - Minimum Node Size
  - Number of Trees
  - Number of Predictors Sampled

# Conclusion

| Feature | **Bagging** | **Boosting** | **Stacking** |
|---|---|---|---|
| **Purpose** | Reduce variance | Reduce bias (and variance) | Improve prediction using multiple models |
| **Model type** | Parallel, independent learners | Sequential, dependent learners | Parallel base models + meta-learner |
| **Data used** | Bootstrap samples | Weighted samples based on errors | Same data for base learners (hold-out for meta) |
| **Combining** | Average / Majority vote | Weighted sum / vote | Meta-model learns combination |
| **Example** | Random Forest | AdaBoost, XGBoost | Any combination (e.g., RF + SVM + NN + meta-model) |

# Disadvantages of Ensemble methods

However, model ensembles are not always better.

- Ensembles can be more difficult to interpret, the output of the ensembled model is hard to predict and explain.
- Require higher computational burden, especially for classifier training